Импорт библиотеки для работы с датафремом и визуализаций

```
import pandas as pd
import matplotlib.pyplot as plt
```

Загрузка датафреймов и краткий обзор

```
df_users = pd.read_csv('practic_users.csv',sep = ',')
df_users.head()
```

	user_id	group	age	city	device_type
0	1001	A	31	Москва	ios
1	1002	A	31	Москва	ios
2	1003	A	25	Москва	android
3	1004	A	20	Москва	ios
4	1005	A	34	Москва	ios

```
df_pay = pd.read_csv('practic_pay.csv',sep = ',')
df_pay.head()
```

	payment_id	user_id	group	step1_opened	step2_entered	step3_confirmed	step4_succe
0	5001	1001.0	A	2024-04-16 06:06:00	2024-04-16 06:10:00	NaN	NaN
1	5002	1002.0	A	2024-04-12 07:38:00	2024-04-12 07:41:00	NaN	NaN
2	5003	1003.0	A	2024-04-26 20:03:00	2024-04-26 20:07:00	NaN	NaN

Объединение таблиц в один датафрейм для удобной работы

```
df = pd.merge(df_users, df_pay, on='user_id', how='left')
```

```
df.head()
```

	user_id	group_x	age	city	device_type	payment_id	group_y	step1_opened	step2_e
0	1001	A	31	Москва	ios	5001.0	A	2024-04-16 06:06:00	2024-0 06:10:0
1	1002	A	31	Москва	ios	5002.0	A	2024-04-12 07:38:00	2024-0 07:41:0
2	1003	A	25	Москва	android	5003.0	A	2024-04-26 20:03:00	2024-0 20:07:0

Обработка строк и создание доп. столбцов для более удобного воспиятия

```
time_columns = ['step1_opened', 'step2_entered', 'step3_confirmed', 'step4_success']
date_columns = ['date_open', 'date_enter', 'date_confirm', 'date_success']

for t_col, d_col in zip(time_columns, date_columns):
    df[d_col] = pd.to_datetime(df[t_col]).dt.date
df = df.drop(columns=time_columns, errors='ignore')

df.head()
```

	user_id	group_x	age	city	device_type	payment_id	group_y	date_open	date_ente
0	1001	A	31	Москва	ios	5001.0	A	2024-04-16	2024-04-16
1	1002	A	31	Москва	ios	5002.0	A	2024-04-12	2024-04-12
2	1003	A	25	Москва	android	5003.0	A	2024-04-26	2024-04-26

Подсчет шагов и групповая статистика

```
df['steps_completed'] = df[date_columns].count(axis=1)

group_stats = df.groupby('group_y')['steps_completed'].agg(['mean', 'median', 'std',
'count', 'min', 'max'])
display(group_stats)
```

	mean	median	std	count	min	max
group_y						
A	2.888379	3.0	1.213827	1308	1	4
B	3.238905	4.0	1.107084	1352	1	4

График № 1 - Динамика пользователей (> 2 шагов)

```
df_filtered = df[df['steps_completed'] > 2]

daily_steps_filtered = df_filtered.groupby(['date_open',
'group_y']).size().reset_index(name='users_with_gt2_steps')
daily_steps_filtered = daily_steps_filtered.rename(columns={'date_open': 'date'})
daily_steps_filtered['date'] = pd.to_datetime(daily_steps_filtered['date'])

plt.figure(figsize=(14, 7))
for group in daily_steps_filtered['group_y'].unique():
    group_data = daily_steps_filtered[daily_steps_filtered['group_y'] ==
group].sort_values('date')
    plt.plot(group_data['date'], group_data['users_with_gt2_steps'],
             marker='o', label=f'Группа {group}', linewidth=2, markersize=8)

plt.title('Количество пользователей, сделавших более 2 шагов, по дням', fontsize=16,
fontweight='bold')
plt.xlabel('Дата', fontsize=12)
plt.ylabel('Количество пользователей (>2 шагов)', fontsize=12)
plt.legend(title='Группа', fontsize=11)
plt.grid(True, alpha=0.3)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

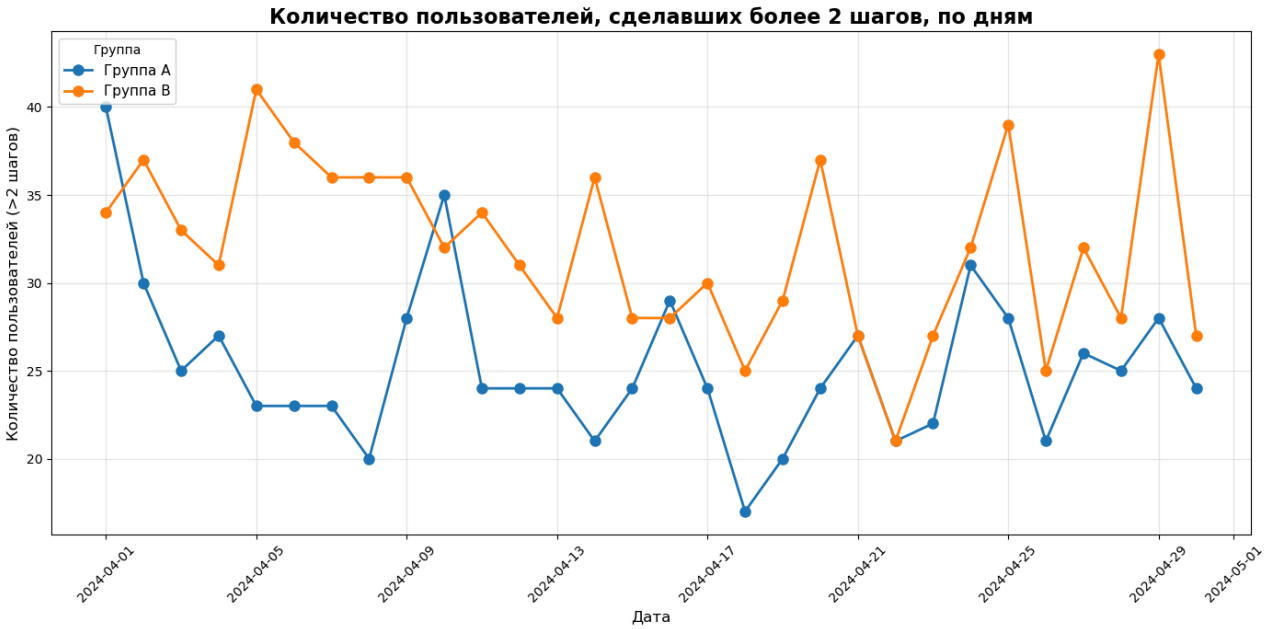


График № 2 - Пользователи, завершившие оплату (4 шага)

```
df_completed = df[df['steps_completed'] == 4]

daily_completed = df_completed.groupby(['date_open',
'group_y']).size().reset_index(name='users_completed')
daily_completed = daily_completed.rename(columns={'date_open': 'date'})
daily_completed['date'] = pd.to_datetime(daily_completed['date'])

plt.figure(figsize=(14, 7))
for group in daily_completed['group_y'].unique():
    group_data = daily_completed[daily_completed['group_y'] ==
group].sort_values('date')
    plt.plot(group_data['date'], group_data['users_completed'],
             marker='o', label=f'Группа {group}', linewidth=2, markersize=8)

plt.title('Количество пользователей, полностью завершивших все 4 шага', fontsize=16,
fontweight='bold')
plt.xlabel('Дата', fontsize=12)
plt.ylabel('Количество пользователей (4 шага)', fontsize=12)
plt.legend(title='Группа', fontsize=11)
plt.grid(True, alpha=0.3)
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

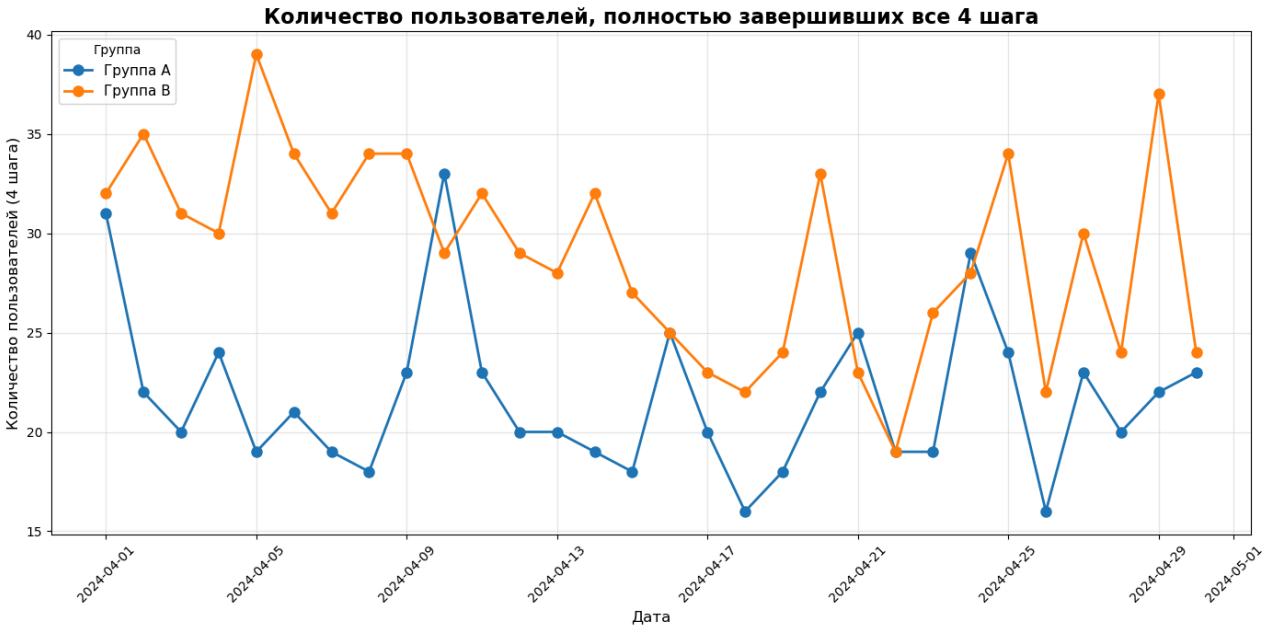


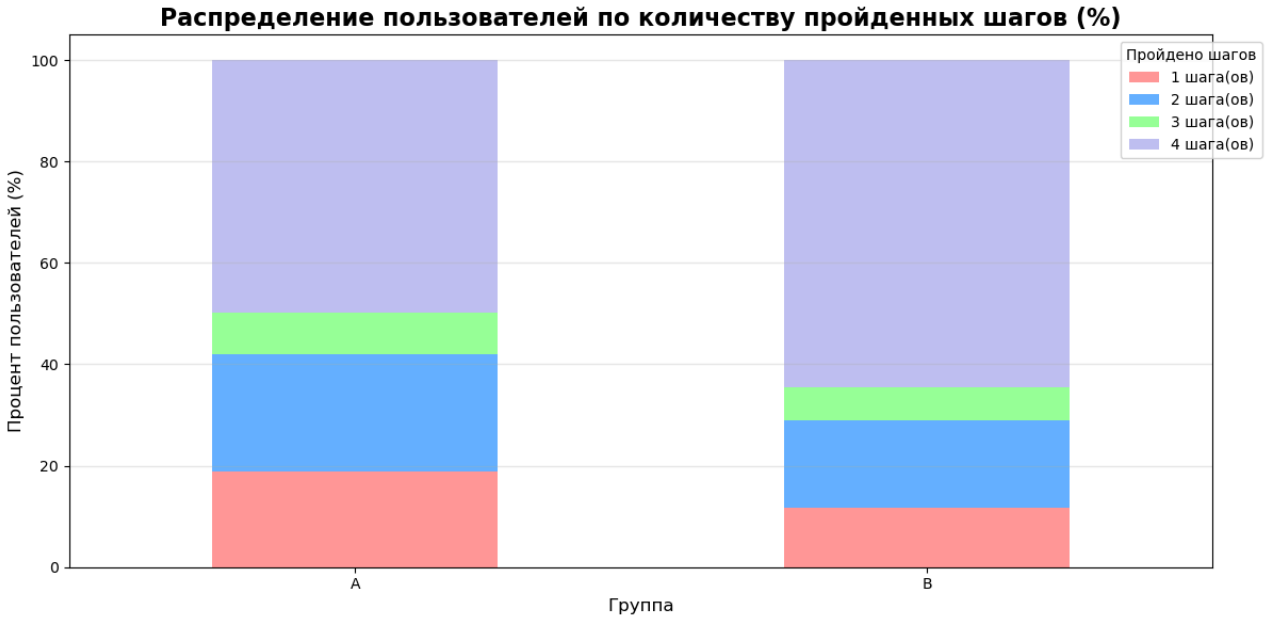
График № 3 - Структура воронки

```
df['steps_cat'] = df['steps_completed'].astype(str) + ' шага(ов)'

pivot = pd.crosstab(df['group_y'], df['steps_cat'], normalize='index') * 100

pivot.plot(kind='bar', stacked=True, figsize=(12, 6),
           color=['#ff9999', '#66b3ff', '#99ff99', '#c2c2f0'])

plt.title('Распределение пользователей по количеству пройденных шагов (%)',
fontsize=16, fontweight='bold')
plt.xlabel('Группа', fontsize=12)
plt.ylabel('Процент пользователей (%)', fontsize=12)
plt.legend(title='Пройдено шагов', bbox_to_anchor=(1.05, 1))
plt.xticks(rotation=0)
plt.grid(axis='y', alpha=0.3)
plt.tight_layout()
plt.show()
```



Вывод

- На графике со 100% накоплением отчетливо видно, что в Группе "*B*" доля пользователей, дошедших до конца воронки, составляет около 65%. В то время как в Группе "*A*" этот показатель находится на уровне 50%.
- Также видно, что в Группе "*B*" существенно снизилась доля пользователей, падающих на 1 и 2 шагах, что напрямую подтверждает эффективность упрощенного заполнения реквизитов.
- График №1 показывает количество пользователей, успешно миновавших этап ввода реквизитов. Оранжевая линия (Группа "*B*") на протяжении почти всего месяца находится стабильно выше синей линии (Группа "*A*").
- Динамика успешных оплат также подтверждает доминацию Группы "*B*". Несмотря на колебания Группа "*B*" стабильно генерирует больше успешных транзакций в день.